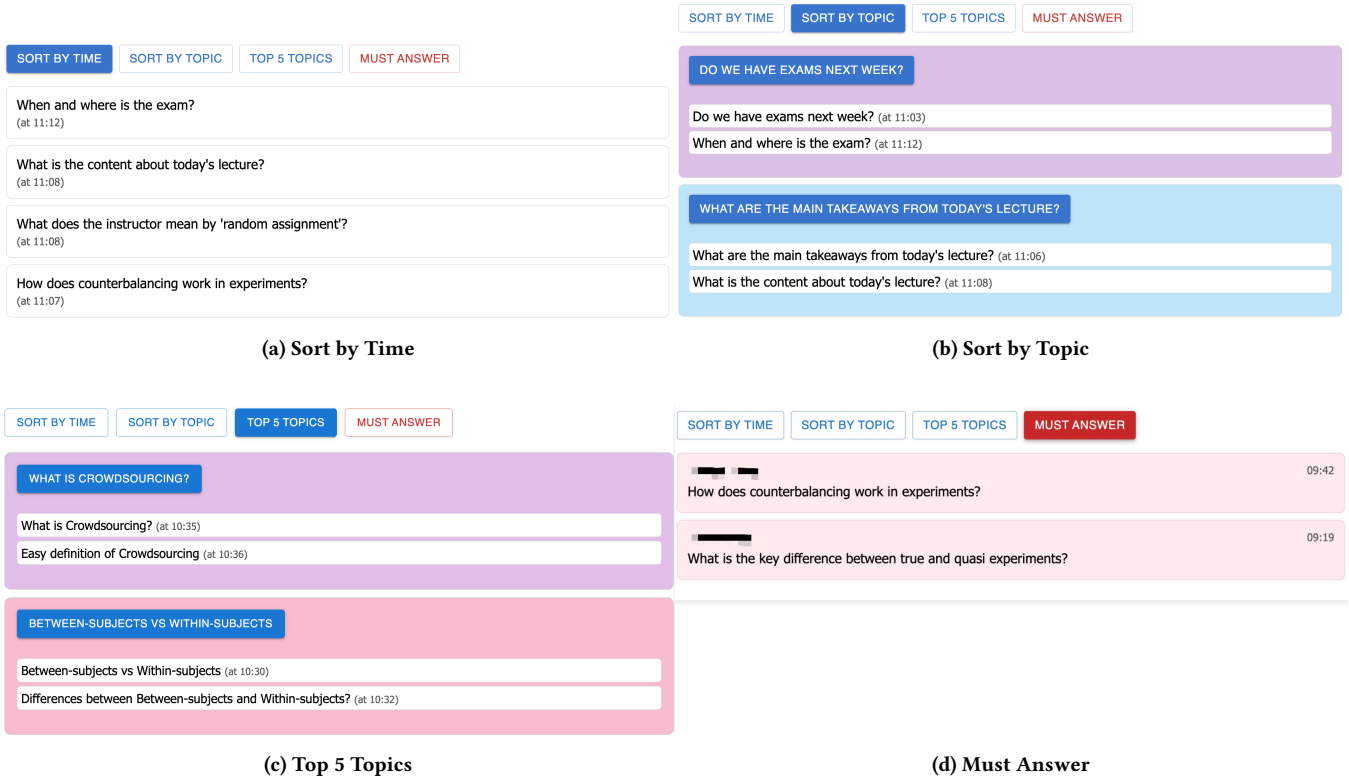


(a) Sort by Time

(b) Sort by Topic

(c) Top 5 Topics

(d) Must Answer

Figure 4: Instructor interface with four modes of question aggregation: (a) Displays recent questions at the top; (b) Clusters semantically similar questions; (c) Highlights the most frequent clusters; (d) Shows questions flagged by students for instructor clarification. Screenshots are taken from the actual deployment of the system. In (d), student names are blurred to preserve anonymity.

This targeted approach helps instructors efficiently prioritize areas needing immediate attention without overwhelming them with redundant or low-value queries.

4. Must Answer. Recognizing the potential limitations of AI-generated responses—particularly as noted by students who sometimes require direct instructor clarification—we introduced a special interaction mode. After receiving AI-generated responses, students may use a dedicated "Send to Instructor for Answer" button if they feel their question remains unresolved or unsatisfactorily answered. Such questions, analogous to traditional hand-raising, are separately collected under the "Must Answer" category, prominently displayed to ensure instructors promptly address these pressing concerns.

These four modes of aggregation collectively provide instructors with actionable, real-time insights into students' shared points of confusion. This design not only addresses the need for scalable monitoring but also ensures that critical questions requiring direct instructor input remain visible and prioritized.

4.4 System Implementation and Technical Details

AskNow is designed as an interactive system with an architecture tailored to the dynamic, high-concurrency, and low-latency requirements of real-time classroom use. In this section, we report the main technical challenges and the corresponding implementation strategies.

4.4.1 Dynamic, Low-Latency Context Management. To precisely understand and utilize a constantly changing lecture context in real time, we developed a decoupled full-stack architecture. The student-facing interface is implemented in Next.js (TypeScript) and deployed on Vercel for elastic, Internet-facing scale, while the instructor interface is hosted privately on cloud servers for secure, real-time monitoring. A Node.js + Express backend on AWS EC2 [1] provides a reliable infrastructure for concurrent requests, with Socket.IO handling real-time bidirectional communication.

We implemented efficient context retrieval through differential updates strategy. Returned transcripts are incrementally materialized on the server as structured, segmented JSON with stable segment indices and timestamps. When answering, it fetches only

[1] https://aws.amazon.com/ec2/